

**Instituto Federal de Educação, Ciência e Tecnologia de São
Paulo
Campus Salto**

Sistemas Operacionais

Projeto de um Núcleo Multitarefas

Salto – São Paulo
2026

Sumário

1. Introdução	5
2. Fases de Desenvolvimento	7
3. Considerações Iniciais	8
4. Fase1: Entendimento das Ferramentas necessárias à Multiprogramação.....	9
4.1 Fase 1: Desenvolvimento e Implementação	9
4.2 Arquivo System.h e System.c.....	9
4.2.1 O arquivo <i>system.h</i> (A Interface).....	9
4.2.2 Arquivo <i>System.c</i>	10
5. Fase 2: Criação das Estruturas.....	14
5.1 Criação das Estruturas Básicas do Núcleo	14
6. Sugestão para Algoritmos das Funções do Núcleo Básico (sem mecanismos de comunicação).....	16
7. Teste do Núcleo Básico	19
8. Implementação de Semáforos.....	22
8.1 Criação de Semáforo	22
8.2 Implementação da Fila de Bloqueados.....	22
8.3 Primitivas para Suporte de Semáforo	23
9. Teste de Implementação de Semáforo	25
9.1 O Problema do Produtor/Consumidor em Buffer Circular.....	25
9.2 O Problema Produtor/Consumidor atuando sobre um Buffer Circular.	25
9.3 Problema Produtor/Consumidor em algoritmo	26

Lista de Figuras

Figura 1: Estrutura do projeto.....	6
Figura 2: Estrutura de dados de system.h.....	9
Figura 2: Estrutura de dados de system.h.....	10
Figura 3: Trecho de código que exemplifica o uso de co-rotinas com fibers.....	11
Figura 4: Funcionamento do Transfer (conhecido como ping-pong entre funções.....	12
Figura 5: Trecho de código fonte exemplo do uso da função tranfer(), implementado com Fiber.....	13
Figura 6: Trecho de código fonte da definição da estrutura de dados desc_P, descreve a descritor de processos.....	14
Figura 7: Trecho de código fonte sugestão para variáveis globais e função para iniciar a fila de processos prontos	16
Figura 8: Algoritmo da Função cria_processo.....	16
Figura 9: Algoritmo da função próximo_ativo_depois().....	17
Figura 10: Lógica da função trampolim	17
Figura 11: Lógica da função dispara_sistema().....	17
Figura 12: Lógica da função yield.....	18
Figura 13: Algoritmo da função termina_processo	18
Figura 14: Exemplo de escalonador	19
Figura 15: exemplo de um processo simples.....	20
Figura 16: Exemplo de programa principal para testar o núcleo.....	20
Figura 17 – Anatomia do Semáforo: Integração entre o contador de sincronização (s) e o encadeamento da fila de processos bloqueados (Q).	22
Figura 18 – Descritor de processo implementando semáforos.....	22
Figura 19 - Primitiva inicia semáforo com tratamento de erro e configuração de estado inicial.	23
Figura 20 – Primitiva da operação P (ou UP).....	23
Figura 21 - Implementação das primitivas de sincronização e gestão da fila de bloqueados interna à estrutura circular do núcleo.	24
Figura 22 – Diagrama ilustrando o problema Produtor/Consumidor com Buffer Circular	25

Lista de Tabelas

Tabela 2: Resumo da Estrutura do Projeto 5

1. Introdução

- **Objetivo:** criar uma máquina virtual para a execução de processos concorrentes, permitindo que os mesmos se sincronizem e se comuniquem.
- **Linguagem de programação utilizada:** linguagem C e suas bibliotecas, utilizando API nativa de *Fibers* para realizar troca de contexto eficiente em espaço de usuário. O escalonamento é cooperativo, onde os processos cedem explicitamente a CPU através da primitiva *yield()*.
- . Não será permitido o uso de biblioteca como *pthread*, *thread*, *semaphore*, etc.
- **Número de alunos:** dois por grupo.
- **Fornecido:** arquivo *System.h* e *System.c*

Na Tabela 1 existe um resumo da estrutura do projeto, com características, atributos, bibliotecas C necessárias, arquivos fornecidos e o solicitado para o projeto.

Tabela 1: Resumo da Estrutura do Projeto

Estruturação do Projeto

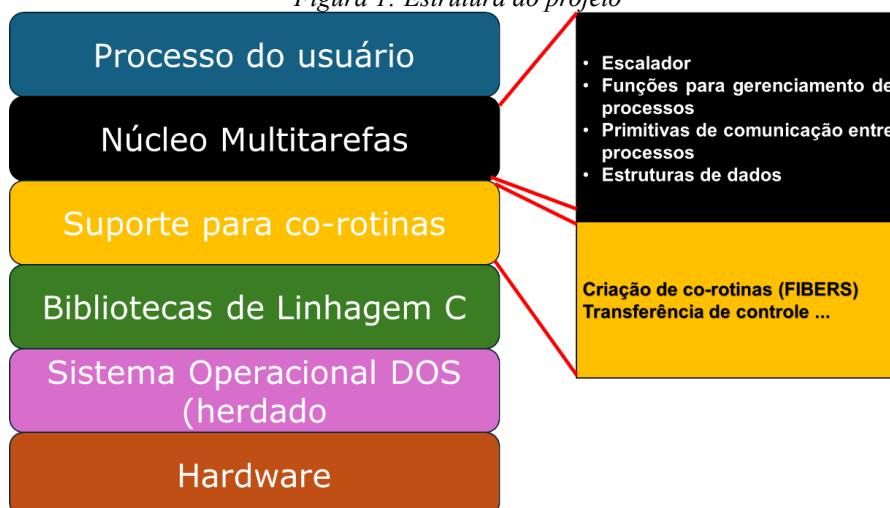
Hardware	Processador x86/x64 (Windows 10/11)	
Windows API	Fibers (SwitchToFiber / CreateFiber)	<i>Você vai ter que estudar</i>
GNU GCC	Compilador C moderno (MinGW-w64)	
Suporte Fibers	system.h / system.c	Dado
Núcleo Multitarefa	nucleo.h / nucleo.c	Solicitado
Semáforos P/V	semaforo.h / semaforo.c	Solicitado
Aplicação	main.c (Produtor/Consumidor)	

Fonte: Elaborado pelo autor (2026)

Na

Figura 1 está a estrutura do projeto em camadas hierárquicas de desenvolvimento, você deve ter atenção especial a camada **Núcleo Multitarefa** e **Suporte para co-rotinas** que é apresentada com um detalhamento.

Figura 1: Estrutura do projeto



Fonte: Elaborado pelo autor (2026)

2. Fases de Desenvolvimento

Você deve atentar as fases de desenvolvimento uma vez que tem em posse o *system.h* e *system.c* que é base para o funcionamento do sistema. As fases de desenvolvimento são:

- Entendimento das ferramentas necessárias ao suporte de multiprogramação.
- Criação das estruturas básicas para funcionamento do núcleo.
- Criação do núcleo multitarefas, oferecendo primitivas Envia e Recebe sinais bloqueantes, como mecanismo de comunicação e sincronização entre processos.
- Implementação das primitivas de semáforos (P e V – Down e UP).
- Teste do núcleo com a programação do problema do produtor/consumidor.

3. Considerações Iniciais

O compartilhamento da UCP neste projeto foi implementado de forma cooperativa (e não preemptiva), tendo como peça central a função ***transfer()*** (*system.h/c*). Em vez de o sistema operacional interromper um processo à força após uma fração de segundo, neste projeto são os próprios processos que precisam decidir quando devem liberar a CPU para os outros. Essa liberação acontece quando um processo cede voluntariamente o controle invocando a função ***yield()***, ou quando fica bloqueado ao tentar consumir um recurso indisponível através da função P() de um semáforo. Nesse momento, o núcleo consulta a fila circular de processos, escolhe o próximo descritor que esteja no estado "ATIVO" e invoca a troca de contexto. O ***transfer()*** então congela instantaneamente o estado do processo atual e carrega o estado do próximo processo no processador.

4. Fase1: Entendimento das Ferramentas necessárias à Multiprogramação

4.1 Fase 1: Desenvolvimento e Implementação

Para a Fase1 se faz necessário para a implementação da Multiprogramação os seguintes elementos:

- Primitivas para manipulação de Co-rotinas (*Fibers*);
- Primitiva para a transformação de uma função C em co-rotina;
- Primitiva para transferência de controle entre co-rotinas, e
- Primitiva para instalação e transferência de controle entre co-rotinas de interrupção.

O código disponível para esta fase encontra-se no arquivo *System.c* (protótipo em *System.h*)

4.2 Arquivo *System.h* e *System.c*

Os arquivos *system.h* e *system.c* formam a camada de abstração de mais baixo nível (Hardware/Sistema Operacional) do seu projeto de núcleo multitarefas. O objetivo principal deles é prover a mecânica básica de troca de contexto, permitindo que o programa pause a execução de uma função e pule para outra, salvando o estado atual para que possa retornar mais tarde. Como a linguagem C padrão não possui suporte nativo para salvar e restaurar pilhas de execução de forma fácil, utiliza-se a uma funcionalidade específica do *Windows* chamada *Fibers* (Fibras).

No *Windows*, *Fibers* são unidades de execução mais leves que as *Threads*. Enquanto as *threads* são agendadas (escalonadas) de forma preemptiva pelo próprio núcleo do *Windows* (o *Windows* decide quem roda e quando), as *Fibers* precisam ser trocadas manualmente pela própria aplicação.

Isso as torna perfeitas para implementar um núcleo de escalonamento cooperativo acadêmico, onde o seu código é quem dita as regras de quem ganha a UCP através de chamadas como *yield()*.

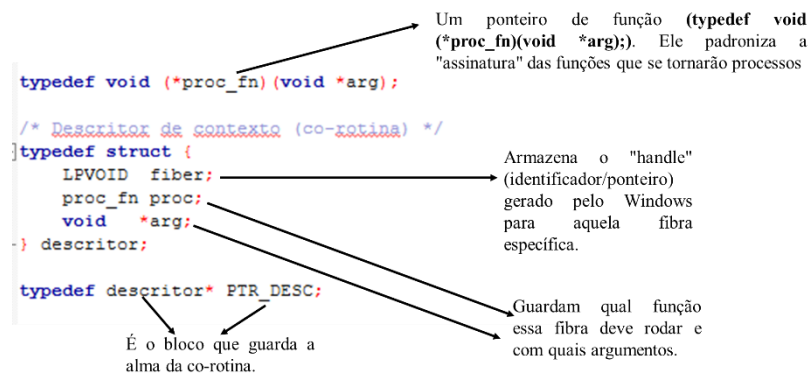
4.2.1 O arquivo *system.h* (A Interface)

Este arquivo define os tipos de dados e os protótipos de função que o *nucleo.c* vai utilizar. Ele "esconde" a complexidade da API do *Windows* do resto do seu sistema.

A estrutura de dados *descriptor* (Figura 2) é utilizado para guardar o contexto na troca de controle entre co-rotinas. Essa a estrutura de dados implementada no *system.h*, apresenta o *descriptor*. A sua função é armazenar o contexto completo de uma co-rotina (neste caso, uma *Fiber* do *Windows*). Basicamente, empacota tudo o que o sistema necessita para criar, pausar e retomar a execução de um processo no seu núcleo.

Figura 2: Estrutura de dados de *system.h*

Figura 3: Estrutura de dados de system.h



Fonte: Elaborado pelo autor (2026)

As funções necessárias e implementadas em *system.c* são:

- ***system_init_main(PTR_DESC d_main)***: Transforma a thread principal do programa (o main do C) em uma fibra. Isso é obrigatório no Windows; uma *thread* normal não pode pular para uma fibra, ela precisa virar uma fibra primeiro.
- ***cria_desc(void)***: Simplesmente aloca dinamicamente (usando malloc ou calloc) memória para um novo struct descritor.
- ***newprocess(proc_fn proc, void *arg, PTR_DESC d)***: Prepara uma nova fibra (co-rotina) vinculando a função (proc), o argumento (arg) e guardando tudo no descritor (d).
- ***transfer(PTR_DESC origem, PTR_DESC destino)***: O "coração" do sistema. É esta função que congela a execução da fibra de origem e retoma a execução da fibra de destino.

4.2.2 Arquivo System.c

O *system.c* contém a implementação real dessas funções. Como o *system.h* faz a importação do <system.h>, o *system.c* atua mapeando as funções do seu núcleo para as funções nativas da API do Windows:

❖ Em *system_init_main*:

- Provavelmente chama a função da API do Windows `ConvertThreadToFiber(NULL)`. O retorno é salvo no `d_main->fiber`.
- **Formato:** `void system_init_main(PTR_DESC d_main);`

❖ Em *cria_desc*:

- Faz um `malloc(sizeof(descritor))`.
- **Formato:** `PTR_DESC cria_desc(void)`

❖ **Em newprocess:**

- Chama a API **CreateFiber(0, LPFIBER_START_ROUTINE)funcao_intermediaria, d)**. Para que esta criação ocorra corretamente, a função newprocess recebe e processa três parâmetros essenciais:
- **proc_fn proc**: O ponteiro para a função C contendo o código que a fibra irá executar (ex: a rotina do produtor ou consumidor).
 - **void *arg**: Um ponteiro genérico contendo possíveis dados de inicialização que a função proc possa necessitar ao começar a rodar.
 - **PTR_DESC d**: O descritor de contexto propriamente dito. O identificador (*handle*) retornado pelo CreateFiber é salvo dentro deste descritor, juntamente com o proc e o arg.
- **Formato:** void newprocess(proc_fn proc, void *arg, PTR_DESC d)

A seguir na Figura 4 se tem um exemplo do uso de co-rotinas com *fibers*.

Figura 4: Trecho de código que exemplifica o uso de co-rotinas com *fibers*

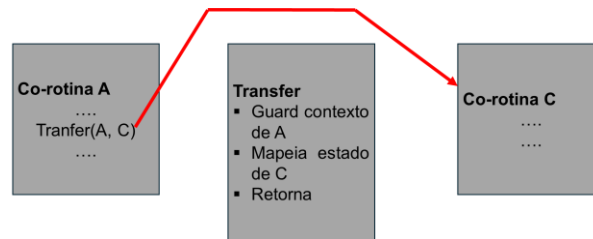
```
1. PTR_DESC d0;
2. ...
3. /* co-rotina adaptada para as restrições da API de Fibers */
4. void corotina0(void *arg) /* código da co-rotina recebe obrigatoriamente um void* */
5. {
6. ...
7. }
8. ...
9. /* programaprincipal */
10. int main()
11. {
12.     d0 = cria_desc(); /* cria descritor de co-rotina */
13.
14.     newprocess(corotina0, NULL, d0); /* cria a co-rotina passando a função, um
    argumento nulo, e o descritor */
15. ...
16. }
17.
```

Fonte: Elaborado pelo autor (2026)

- ❖ **Em transfer:** Chama a API **SwitchToFiber(destino->fiber)**. No momento em que essa linha executa, o processador salva os registradores do processo atual e carrega os registradores do próximo, pulando para o código do outro processo. A função transfer só retorna quando (e se) algum outro processo fizer um *transfer* de volta para ela.

Na Figura 5 é ilustrado o funcionamento da função *transfer*, que em outras palavras se pode ilustra-la como um ping-pong entre funções.

Figura 5: Funcionamento do Transfer (conhecido como ping-pong entre funções)



Fonte: Elaborado pelo autor (2026)

O *núcleo* do sistema atua como um grande "maestro" lógico, enquanto o módulo *system* funciona como o seu "motor" físico. Em linhas gerais, a integração ocorre da seguinte forma para tornar a multitarefa possível:

- **Delegação da Criação:** O núcleo, por si só, não sabe *como* salvar ou isolar a memória de um novo processo. Portanto, toda vez que uma nova tarefa precisa ser cadastrada no sistema, o núcleo delega ao *system* a responsabilidade de alocar e preparar os bastidores físicos (como a pilha de execução) para essa tarefa.
- **O Início do Show:** Quando o programa principal termina as preparações e decide iniciar a simulação contínua, o núcleo solicita ao *system* que engate o programa base nessa mesma mecânica de troca e inicie imediatamente a execução do primeiro processo que estiver na fila de espera.
- **A Dança da Troca de Contexto:** Esta é a principal integração que ocorre repetidamente durante o funcionamento do projeto. Sempre que um processo ativo decide ceder a sua vez voluntariamente ou é obrigado a aguardar porque um recurso ainda não está pronto, o núcleo assume o controle. Ele consulta suas filas, toma a decisão de *quem* deve ser o próximo a executar e, então, aciona a ferramenta de transferência do *system*. É nesse momento que o *system* realiza a mágica física instantânea no processador: congela o processo atual e devolve o controle da CPU para o próximo processo escolhido.

Abaixo, na Figura 6, se tem o seu exemplo adaptado usando as exigências de *Fibers* (assinatura com `void *arg`, uso de `NULL` em *newprocess* e inicialização mandatória do *main* via `system_init_main`):

Figura 6: Trecho de código fonte exemplo do uso da função `transfer()`, implementado com Fiber

```
1. #include "system.h"
2. #include <stdio.h>
3. #include <stdlib.h>
4.
5. PTR_DESC c01, c02, prin;
6.
7. /* co-rotinas (devem aceitar um void *arg) */
8. void corotina1(void *arg)
9. {
10.     while(1)
11.     {
12.         printf("tic- ");
13.         transfer(c01, c02); /* Congela c01, pula para c02 */
14.     }
15. }
16.
17. void corotina2(void *arg)
18. {
19.     while(1)
20.     {
21.         printf("tac\n");
22.         transfer(c02, c01); /* Congela c02, pula de volta para c01 */
23.     }
24. }
25.
26. /* Programa Principal */
27. int main(void)
28. {
29.     system("cls"); /* Alternativa moderna ao clrscr() */
30.
31.     /* cria descritores de co-rotinas */
32.     c01 = cria_desc();
33.     c02 = cria_desc();
34.     prin = cria_desc();
35.
36.     /* associa descritores com as co-rotinas */
37.     /* Na API do projeto, o argumento do meio é um void* para parâmetros. Passamos NULL.
38.     */
39.     newprocess(corotina1, NULL, c01);
40.     newprocess(corotina2, NULL, c02);
41.     /* OBRIGATÓRIO: Converte o fluxo principal do Windows em uma Fiber antes de fazer a
42.     transferência */
43.     system_init_main(prin);
44.
45.     /* transfere o controle da thread/fiber principal para a co-rotina1 */
46.     transfer(prin, c01);
47.     return 0;
48. }
49.
```

Fonte: Elaborado pelo autor (2026)

5. Fase 2: Criação das Estruturas

5.1 Criação das Estruturas Básicas do Núcleo

O núcleo terá uma estrutura de dados chamada *desc_p*, para que o núcleo multitarefas consiga gerenciar o revezamento da CPU, cada processo criado deve ter um descritor de processo associado.

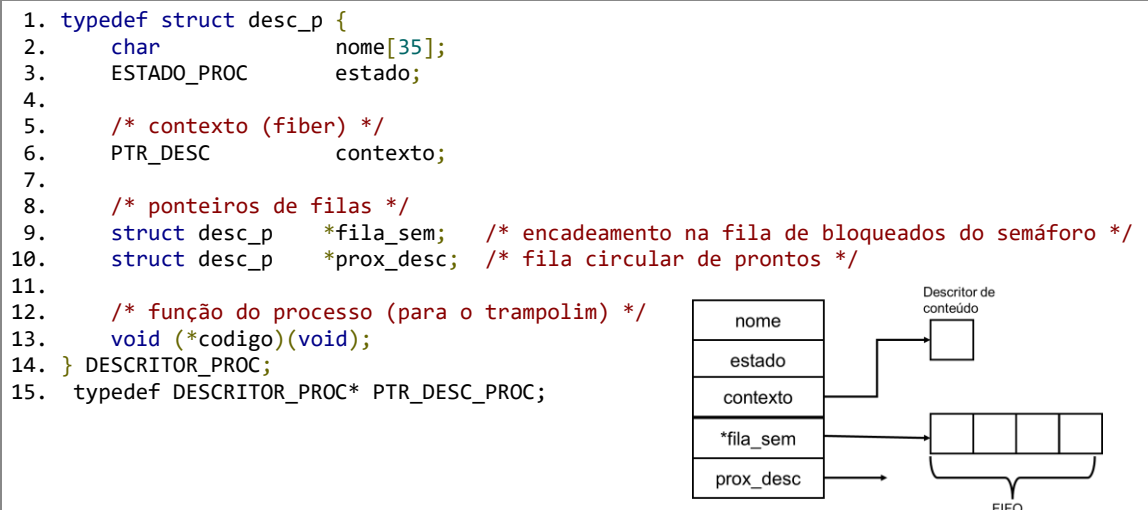
O descritor atua como o Bloco de Controle do Processo (PCB - *Process Control Block*) da nossa implementação. Ele terá os seguintes campos:

- **Nome do processo (char nome[35]):** Uma *string* para identificar o processo de forma legível (ex: "Produtor" ou "Consumidor"), facilitando o rastreamento e o debug.
- **Estado do processo (ESTADO_PROC estado):** Um enumerador que dita a situação atual do processo no escalonador. Pode estar ATIVO (apto a rodar), BLOQ_P (aguardando liberação de semáforo) ou TERMINADO (função concluída).
- **Ponteiro para descritor de contexto (PTR_DESC contexto):** A ponte que liga o núcleo lógico à camada física. É a referência para o descritor base definido em system.h, que guarda o *handle* da *Fiber* do Windows para a troca de registradores.
- **Ponteiro para o próximo descritor (struct desc_p *prox_desc):** O elo de encadeamento da Fila Circular de prontos. O escalonador utiliza este ponteiro para varrer a lista e passar o controle da CPU em *Round-Robin* (passando a vez adiante).

```
Na 1. typedef struct desc_p {
2.     char        nome[35];
3.     ESTADO_PROC estado;
4.
5.     /* contexto (fiber) */
6.     PTR_DESC     contexto;
7.
8.     /* ponteiros de filas */
9.     struct desc_p *fila_sem; /* encadeamento na fila de bloqueados do semáforo */
10.    struct desc_p *prox_desc; /* fila circular de prontos */
11.
12.    /* função do processo (para o trampolim) */
13.    void (*codigo)(void);
14. } DESCRITOR_PROC;
15. typedef DESCRITOR_PROC* PTR_DESC_PROC;
```

se pode ver a ideia de implementação da estrutura de dados par ao descritor de processos do núcleo.

Figura 7: Trecho de código fonte da definição da estrutura de dados *desc_P*, descreve a descritor de processos



Fonte: Elaborado pelo autor (2026)

Além desses campos básicos iniciais, a implementação evoluiu e consolidou mais dois atributos vitais para a sincronização:

- **Ponteiro para a fila de semáforos (struct desc_p *fila_sem):** Responsável por encadear o processo na fila de espera específica de um semáforo quando ele executa a função P() e o recurso está zerado.
- **Ponteiro de função do usuário (void (*codigo)(void)):** Armazena a rotina lógica original do processo. Este ponteiro é executado indiretamente por uma função "trampolim" no núcleo, que garante que, se a rotina do usuário chegar ao fim (um *return* implícito), a função *termina_processo()* seja invocada automaticamente para limpar o estado.

Componentes e funções Básicas para o núcleo:

- **cria_processo (...):** associa um descritor de processo ao código do processo e coloca-o na fila dos prontos;
- **inicia_fila_prontos():** inicia o cabeça da fila dos prontos (variável global *prim*);
- **dispara_sistema():** transfere o controle do programa principal (*main*) para o escalador;
- **escalador() :** co-rotina do escalador.
- **termina_processo():** marca o processo chamador como “terminado”;

Como **sugestão** se pode ter mais algumas funções, como:

- **cria_processo (...):** associa um descritor de processo ao código do processo e coloca-o na fila dos prontos;
- **inicia_fila_prontos():** inicia o cabeça da fila dos prontos (variável global *prim*);
- **dispara_sistema():** transfere o controle do programa principal (*main*) para o escalador;
- **escalador() :** co-rotina do escalador.
- **termina_processo():** marca o processo chamador como “terminado”;

6. Sugestão para Algoritmos das Funções do Núcleo Básico (sem mecanismos de comunicação)

Aqui é onde as variáveis globais ganham efetivamente espaço na memória. também instanciamos o contexto especial para a função *main* e será implementado a rotina de inicialização *inicia_fila_prontos()*.

O trecho (Figura 8) a seguir demonstra na prática a declaração dos ponteiros essenciais para o rastreamento da fila circular de processos (*prim* e *atual*), a criação isolada do contexto da thread principal (*main_ctx*) e a execução da função de preparo inicial, responsável por garantir que o núcleo inicie com um estado totalmente limpo e seguro:

Figura 8: Trecho de código fonte sugestão para variáveis globais e função para iniciar a fila de processos prontos

```
1. /* Fila circular (Variáveis reais) */
2. PTR_DESC_PROC prim = NULL;
3. PTR_DESC_PROC atual = NULL;
4.
5. /* Contexto do main (como fiber - substitui o conceito do antigo "d_esc") */
6. static descriptor main_desc;
7. static PTR_DESC main_ctx = &main_desc;
8. static int main_ready = 0;
9.
10. /* ... outras funções auxiliares como processo_trampoline ... */
11.
12. /* Função Básica de Preparação */
13. void inicia_fila_prontos(void)
14. {
15.     prim = NULL;
16.     atual = NULL;
17. }
18.
```

Fonte: Elaborado pelo autor (2026)

A função **cria_processo** é responsável por instanciar novas tarefas no núcleo. Ela aloca memória, configura o Bloco de Controle do Processo (PCB) e o insere na fila circular de escalonamento (Figura 9).

Figura 9: Algoritmo da Função *cria_processo*

```
1. void cria_processo(void (*end_proc)(void), const char *nome_p)
2. {
3.     - cria descritor (tipo DESCRITOR_PROC) dinamicamente via malloc;
4.     - inicia os campos do descritor:
5.         - copia (strncpy) nome_p para o campo nome;
6.         - marca o estado como ATIVO;
7.         - guarda a função original (end_proc) no campo 'codigo';
8.         - inicializa fila_sem como NULL;
9.     - cria o descritor de contexto físico (cria_desc);
10.    - inicia descritor de contexto vinculando a Fiber ao trampoline:
11.        (newprocess(processo_trampoline, novo_descritor, contexto));
12.    - insere descritor de processo no final da fila circular dos prontos;
```

Fonte: Elaborado pelo autor (2026)

A função **proximo_ativo_depois** (Figura 10) é o mecanismo de busca que permite ao núcleo localizar a próxima tarefa pronta para execução. Ela percorre a fila circular a partir de um ponto específico, garantindo que todos os processos tenham a oportunidade de receber a CPU.

Figura 10: Algoritmo da função **proximo_ativo_depois()**

```
1. static PTR_DESC_PROC proximo_ativo_depois(PTR_DESC_PROC a_partir)
2. {
3.     - verifica se a fila circular (prim) existe e não está vazia;
4.     - define o ponto inicial da busca (usa prim se a_partir for nulo);
5.     - inicia a varredura a partir do próximo elemento da lista (prox_desc);
6.     - percorre a lista circular enquanto não retornar ao ponto de partida original:
7.         - se encontrar um processo com estado igual a ATIVO, retorna este descritor;
8.         - avança para o próximo ponteiro (prox_desc);
9.     - após completar a volta completa, verifica se o próprio ponto de partida está ATIVO;
10.    - se nenhum processo em toda a fila estiver apto, retorna NULL;
11. }
12.
```

Fonte: Elaborado pelo autor (2026)

A função **processo_trampoline** atua como um invólucro de segurança para a execução das tarefas. Ela garante que, após a conclusão natural da função do usuário, o controle retorne ao núcleo para encerramento. A lógica estrutural desta função segue o algoritmo descrito na Figura 11.

Figura 11: Lógica da função **processo_trampoline**

```
1. static void processo_trampoline(void *arg)
2. {
3.     - converte o argumento genérico (void *arg) para o tipo real (PTR_DESC_PROC);
4.     - valida se o descritor recebido e o ponteiro de código são válidos;
5.     - invoca a função lógica do processo armazenada no campo 'codigo';
6.     - após o retorno da função (término do código do usuário), chama 'termina_processo()';
7.     - assegura que o processo seja marcado como TERMINADO e a CPU seja cedida ao próximo.
8. }
9.
```

Fonte: Elaborado pelo autor (2026)

A função **dispara_sistema** é o ponto de entrada para a execução multitarefa. Ela prepara o contexto do sistema operativo e inicia o ciclo de alternância entre os processos ativos. O algoritmo para o disparo do sistema segue na Figura 12.

Figura 12: Lógica da função **dispara_sistema()**

```
1. void dispara_sistema(void)
2. {
3.     - verifica se existem processos criados na fila (prim);
4.     - inicializa o contexto da função main como uma fiber (system_init_main);
5.     - marca o sistema como pronto (main_ready) para permitir retornos futuros;
6.     - seleciona o primeiro processo pronto para execução:
7.         - se 'prim' estiver em estado ATIVO, seleciona 'prim';
8.         - caso contrário, utiliza 'proximo_ativo_depois(prim)' para encontrar um candidato;
9.     - valida se um processo ativo foi efetivamente encontrado;
10.    - atualiza o ponteiro global 'atual' com o processo selecionado;
11.    - realiza a transferência física de contexto (transfer) do main_ctx para o atual->contexto;
12.    - a partir deste ponto, o núcleo assume o controle cooperativo até que todas as tarefas terminem.
13. }
14.
```

Fonte: Elaborado pelo autor (2026)

A função *yield* permite que o processo atualmente em execução ceda voluntariamente a CPU para que outro processo apto possa correr, mantendo o princípio da multitarefa cooperativa. O algoritmo para o revezamento segue na Figura 13.

Figura 13: Lógica da função *yeld*

```
1. void yield(void)
2. {
3.   - verifica se existe um processo 'atual' devidamente inicializado;
4.   - invoca a busca pelo 'proximo_ativo_depois' a partir do processo que está a ceder a vez;
5.   - valida o resultado da busca:
6.     - se não houver outro processo no estado ATIVO (ou se o próximo for o próprio atual), a função termina e a execução continua no mesmo processo;
7.   - armazena o processo que está a sair numa variável temporária ('antigo');
8.   - atualiza o ponteiro global 'atual' para o novo descritor selecionado;
9.   - executa a função 'transfer' para salvar o estado do processo antigo e carregar o estado do novo;
10.  - o processo que cedeu a vez permanecerá pausado até ser novamente escolhido pelo escalonador.
11. }
12.
```

Fonte: Elaborado pelo autor (2026)

A função *termina_processo* é invocada (geralmente pelo trampolim) quando um processo conclui a sua execução. Ela marca o processo como finalizado e garante que a CPU não fique ociosa, transferindo o controlo para o próximo processo ativo ou de volta para o programa principal. O algoritmo para o encerramento segue na Figura 14.

Figura 14: Algoritmo da função *termina_processo*

```
1. void termina_processo(void)
2. {
3.   - verifica se existe um processo 'atual' em execução;
4.   - altera o estado do processo 'atual' para TERMINADO;
5.   - invoca a busca pelo 'proximo_ativo_depois' a partir do processo atual para encontrar outra tarefa apta;
6.   - se um novo processo ativo for encontrado (diferente do atual):
7.     - armazena o descritor do processo atual num ponteiro temporário ('antigo');
8.     - atualiza o ponteiro global 'atual' para o novo processo selecionado;
9.     - executa a função 'transfer' para carregar o contexto do novo processo e libertar o antigo;
10.  - caso contrário (não existirem mais processos ativos na fila circular):
11.    - verifica se o sistema está pronto para retornar ao programa principal ('main_ready');
12.    - se estiver pronto, executa a função 'transfer' para devolver o controlo ao 'main_ctx';
13.    - se o 'main' não estiver acessível, reporta um erro crítico de encerramento do sistema.
14. }
15.
```

Fonte: Elaborado pelo autor (2026)

7. Teste do Núcleo Básico

Após a consolidação dos algoritmos fundamentais, o projeto entra na fase de validação operacional. Esta etapa é crucial para confirmar a estabilidade do escalonamento cooperativo.

- **Implementação dos Algoritmos Anteriores (em Linguagem C):** Integração total entre os módulos system.c (gestão física de fibras) e nucleo.c (gestão lógica de processos).
- **Implementação de Processos Simples:** Criação de tarefas elementares que apenas imprimem mensagens e cedem a CPU (ex: tarefas trampolim").
- **Teste do Escalonador:** Verificação da alternância correta entre processos ativos, garantindo que a fila circular seja percorrida integralmente e que o sistema retorne ao main apenas após a conclusão de todas as tarefas.

Na Figura 15 tem um exemplo de implementação do escalonador.

Figura 15: Exemplo de escalonador

```
1. /**
2.  * 1. O DISPARO (Início do Escalonamento)
3.  * Corresponde à parte do código antigo que preparava d_esc e prim.
4.  */
5. void dispara_sistema(void) {
6.     if (prim == NULL) return;
7.
8.     // Prepara o contexto de "origem" (o main_ctx)
9.     system_init_main(main_ctx);
10.
11.    // Seleciona o primeiro processo destino (equivalente ao prim->contexto)
12.    atual = (prim->estado == ATIVO) ? prim : proximo_ativo_depois(prim);
13.
14.    if (atual != NULL) {
15.        // Faz a transferência inicial (Equivalente ao primeiro iotransfer)
16.        transfer(main_ctx, atual->contexto);
17.    }
18. }
19.
20. /**
21.  * 2. O REVEZAMENTO (A lógica interna do While antigo)
22.  * No código antigo, o while(1) rodava a cada interrupção.
23.  * No atual, esta função é chamada pelos processos (yield).
24.  */
25. void yield(void) {
26.     PTR_DESC_PROC prox;
27.
28.     if (atual == NULL) return;
29.
30.     // Busca o próximo candidato (Equivalente ao procura_proximo_ativo)
31.     prox = proximo_ativo_depois(atual);
32.
33.     // Se houver alguém novo para rodar
34.     if (prox != NULL && prox != atual) {
35.         PTR_DESC_PROC antigo = atual;
36.         atual = prox;
37.
38.         // Troca o contexto (A essência do iotransfer sem precisar de disable/enable)
39.         // O estado de 'antigo' é salvo e o de 'atual' é carregado.
40.         transfer(antigo->contexto, atual->contexto);
41.     }
42. }
```

```

43.
44. /**
45.  * 3. BUSCA ROUND-ROBIN
46.  * Esta função implementa a lógica de procura_proximo_ativo()
47.  */
48. static PTR_DESC_PROC proximo_ativo_depois(PTR_DESC_PROC a_partir) {
49.     PTR_DESC_PROC aux = a_partir->prox_desc;
50.
51.     // Varre a fila circular até achar um ATIVO ou voltar ao início
52.     while (aux != a_partir) {
53.         if (aux->estado == ATIVO) {
54.             return aux;
55.         }
56.         aux = aux->prox_desc;
57.     }
58.
59.     // Se ninguém mais estiver ativo, verifica se o ponto de partida ainda está
60.     return (a_partir->estado == ATIVO) ? a_partir : NULL;
61. }

```

Fonte: Elaborado pelo autor (2026)

Na Figura 16 há um exemplo de tarefas básicas para validar a alternância de contexto. Os processos imprimem caracteres e cedem a UCP, permitindo observar o escalonamento circular e a funcionalidade do núcleo cooperativo.

Figura 16: exemplo de um processo simples

```

1. void processo1()
2. {
3.     while(1)
4.     {
5.         printf("1");
6.         yield(); /* Cede a vez voluntariamente para o próximo da fila */
7.     }
8. }
9.
10. void processo2()
11. {
12.     while(1)
13.     {
14.         printf("2");
15.         yield(); /* Cede a vez voluntariamente para o próximo da fila */
16.     }
17. }

```

Fonte: Elaborado pelo autor (2026)

Na Figura 17 é demonstrado a inicialização do núcleo, a criação de dois processos básicos e o disparo do escalonador, garantindo que o controle da CPU seja transferido corretamente para o sistema multitarefa.

Figura 17: Exemplo de programa principal para testar o núcleo

```

1. /* Programa Principal */
2. int main(void)
3. {
4.     /* 1. Prepara as estruturas internas do núcleo (fila circular) */
5.     inicia_fila_prontos();
6.
7.     /* 2. Instancia os processos no sistema vinculando código e nome */
8.     /* Cada chamada aloca um descritor e insere na fila de prontos */
9.     cria_processo(processo1, "proc1");

```

```
10.     cria_processo(processo2, "proc2");
11.
12.     printf("Sistema pronto. Iniciando escalonador cooperativo...\n");
13.
14.     /* 3. Converte o main em Fiber e transfere o controle para o núcleo */
15.     /* A partir daqui, o código do main fica pausado até todos os processos terminarem */
16.     dispara_sistema();
17.
18.     printf("\nSistema finalizado com sucesso.\n");
19.     return 0;
20. }
```

Fonte: Elaborado pelo autor (2026)

8. Implementação de Semáforos

Deve-se implementar o semáforo segundo o mecanismo proposto por Dijkstra, ou seja, o semáforo deve ser associado a uma fila Q_i^1 a cada semáforo s_i^2 .

8.1 Criação de Semáforo

Na Figura 18 se encontra a estrutura que define o semáforo do sistema, contendo um contador inteiro para controle de recursos e um ponteiro para a fila de processos bloqueados que aguardam a liberação do recurso.

Figura 18 – Anatomia do Semáforo: Integração entre o contador de sincronização (s) e o encadeamento da fila de processos bloqueados (Q).



Fonte: Elaborado pelo autor (2026)

8.2 Implementação da Fila de Bloqueados

Para otimizar o desempenho do núcleo e reduzir a sobrecarga de manipulação de ponteiros na fila circular *prim*, a nossa implementação adota uma estratégia de "sublista" lógica. Em vez de remover fisicamente os descritores da fila de prontos quando um processo é bloqueado ao executar a primitiva P sobre um semáforo zerado — o que exigiria re-inserções dispendiosas após um V — optou-se por manter o descritor permanentemente na fila circular, alterando apenas o seu campo *estado* para **BLOQ_P**. Esta arquitetura exigiu a inclusão do campo específico *fila_sem* na estrutura **DESCRITOR_PROC**, permitindo que um processo seja encadeado na fila de espera Q do semáforo enquanto permanece na lista global. Consequentemente, o escalonador foi desenhado para, durante a sua varredura em *Round-Robin*, simplesmente saltar qualquer nó que não esteja no estado **ATIVO**, garantindo uma gestão de processos mais fluida e segura.

Na Figura 19 o novo descritor ficará no formato da

Figura 19 – Descritor de processo implementando semáforos

```
1. typedef enum {
2.     ATIVO,
3.     BLOQ_P,
4.     TERMINADO
5. } ESTADO_PROC;
6.
7. typedef struct desc_p {
```

¹ Fila de processos em estado de pronto que aguardam a UCP estar livre

² Variável que representa o semáforo, i significa que você ter vários semáforos

```

8.     char                nome[35];
9.     ESTADO_PROC        estado;
10.
11.     /* contexto (fiber) */
12.     PTR_DESC             contexto;
13.
14.     /* ponteiros de filas */
15.     /* encadeamento na fila de bloqueados do semáforo */
16.     struct desc_p        *fila_sem;
17.     /* fila circular de prontos */
18.     struct desc_p        *prox_desc;
19.
20.     /* função do processo (para o trampolim) */
21.     void (*codigo)(void);
22. } DESCRITOR_PROC;
23.

```

Fonte: Elaborado pelo autor (2026)

8.3 Primitivas para Suporte de Semáforo

A função de inicialização configura o estado interno do semáforo, estabelecendo o limite de acessos simultâneos e limpando a fila de espera, permitindo uma sincronização segura entre as tarefas do sistema.

Figura 20 - Primitiva inicia semáforo com tratamento de erro e configuração de estado inicial.

```

1. void inicia_semaforo(semaphore *sem, int n)
2. {
3.     if (!sem) return;
4.     sem->s = n;
5.     sem->q = NULL;
6. }
7.

```

Fonte: Elaborado pelo autor (2026)

A operação **P (Down)** (Figura 21) gerencia a requisição de recursos: se disponíveis, o processo prossegue após decrementar o contador; se não, ele é suspenso e inserido na fila de espera do semáforo.

Diferente de sistemas preemptivos, a nossa implementação dispensa a desativação de interrupções, pois a troca de contexto só ocorre se o recurso estiver indisponível, garantindo a atomicidade da operação.

Figura 21 – Primitiva da operação P (ou UP)

```

1. void P(semaphore *sem)
2. {
3.     PTR_DESC_PROC aux;
4.     PTR_DESC_PROC prox;
5.
6.     if (!sem) return;
7.     if (!atual) {
8.         fprintf(stderr, "P: nenhum processo atual.\n");
9.         exit(1);
10.    }
11.
12.    if (sem->s > 0) {
13.        sem->s--;
14.        return;
15.    }
16.
17.    /* bloqueia o processo atual */

```



```

18.     atual->estado = BLOQ_P;
19.
20.     /* insere no fim da fila do semáforo */
21.     atual->fila_sem = NULL;
22.     if (!sem->Q) {
23.         sem->Q = atual;
24.     } else {
25.         aux = sem->Q;
26.         while (aux->fila_sem) aux = aux->fila_sem;
27.         aux->fila_sem = atual;
28.     }
29.
30.     /* precisa existir alguém ativo para rodar, senão deadlock */
31.     prox = NULL;
32.     if (prim) {
33.         /* procura um ativo a partir do atual */
34.         PTR_DESC_PROC scan = atual->prox_desc;
35.         while (scan && scan != atual) {
36.             if (scan->estado == ATIVO) { prox = scan; break; }
37.             scan = scan->prox_desc;
38.         }
39.         if (!prox && atual->estado == ATIVO) prox = atual;
40.     }
41.
42.     if (!prox) {
43.         fprintf(stderr, "Deadlock: todos os processos estao bloqueados em P().\n");
44.         exit(1);
45.     }
46.
47.     /* cede CPU para outro processo ativo */
48.     yield();
49.
50.     /* quando for acordado por V(), retorna e continua */
51. }
52.

```

Fonte: Elaborado pelo autor (2026)

A operação **V (Up)** (Figura 22) libera recursos: incrementa o contador ou desperta o primeiro processo bloqueado na fila, devolvendo-o ao estado ativo.

Figura 22 - Implementação das primitivas de sincronização e gestão da fila de bloqueados interna à estrutura circular do núcleo.

```

1.  /* Primitiva da operação V (ou Up) */
2.  void V(semáforo *sem)
3.  {
4.      PTR_DESC_PROC p;
5.
6.      if (!sem) return;
7.
8.      /* Se a fila de espera Q estiver vazia, incrementamos o contador s */
9.      if (sem->Q == NULL) {
10.         sem->s++;
11.     } else {
12.         /* Retira o primeiro processo (p) da fila de bloqueados Q */
13.         p = sem->Q;
14.         sem->Q = p->fila_sem;
15.         p->fila_sem = NULL;
16.
17.         /* Muda o estado deste processo para ATIVO */
18.         /* Na próxima varredura do escalonador, este processo poderá rodar */
19.         p->estado = ATIVO;
20.     }
21.
22.     /* Nota: Diferente de sistemas preemptivos, o processo atual não perde

```

```
23.         a CPU imediatamente. Ele continua até chamar yield() ou P(). */
24.     }
```

Fonte: Elaborado pelo autor (2026)

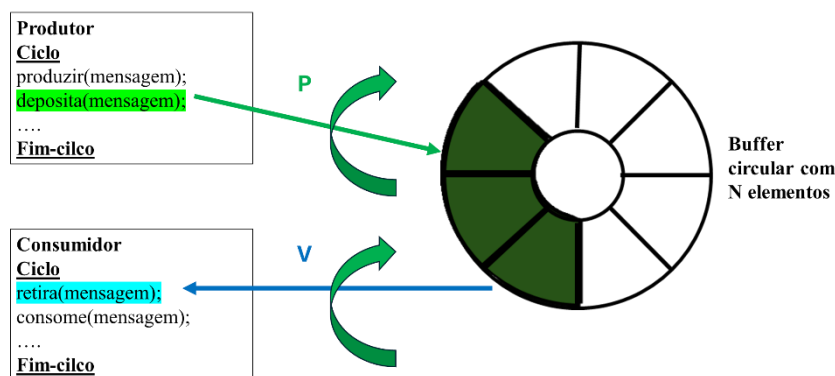
9. Teste de Implementação de Semáforo

A etapa final de validação do núcleo multitarefa consiste na aplicação prática das primitivas de sincronização para resolver problemas clássicos de concorrência.

9.1 O Problema do Produtor/Consumidor em Buffer Circular

A fim de atestar a funcionalidade e a confiabilidade das primitivas de sincronização desenvolvidas, o projeto utiliza a implementação do **Problema do Produtor/Consumidor** operando sobre um **Buffer Circular** (Figura 23). Este cenário de teste é fundamental para observar o comportamento do núcleo frente a condições de disputa por recursos, validando o mecanismo de bloqueio (BLOQ_P) e o despertar de processos (ATIVO) através das operações de semáforo.

Figura 23 – Diagrama ilustrando o problema Produtor/Consumidor com Buffer Circular



Fonte: Elaborado pelo autor (2026)

A simulação demonstra a interação harmônica entre três semáforos distintos:

1. **Mutex**: Garante a exclusão mútua durante a manipulação dos índices do buffer (Região Crítica).
2. **Vazio (Empty)**: Controla a disponibilidade de espaços livres para o Produtor.
3. **Cheio (Full)**: Sinaliza a existência de itens prontos para o Consumidor.

Esta implementação comprova que o escalonador cooperativo, em conjunto com a gestão de filas de bloqueados interna aos semáforos, é capaz de manter a integridade dos dados e a fluidez da execução multitarefa sem a necessidade de preempção por hardware.

9.2 O Problema Produtor/Consumidor atuando sobre um Buffer Circular.

É necessário ter atenção para algumas restrições que este problema apresenta>

- A. o produtor não deve exceder a capacidade finita do buffer;
- B. o consumidor não poderá consumir mensagens mais rapidamente do que forem produzidas;
- C. as mensagens devem ser retiradas do buffer na mesma ordem que forem colocadas, e
- D. restrição de exclusão mútua no acesso ao buffer circular.

9.3 Problema Produtor/Consumidor em algoritmo

1. Algoritmo de Escalonamento (Busca do Próximo Ativo)

Função: proximo_ativo_depois(referencia)

1. **Início**
2. **Se** a fila global (prim) não existe, **Então Retorne** NULO.
3. **Se** a referência for NULA, **Então** referência = prim.
4. Definir auxiliar como o próximo do processo de referência.
5. **Enquanto** auxiliar for diferente da referência:
 - **Se** o estado do auxiliar for igual a ATIVO, **Então Retorne** auxiliar.
 - auxiliar recebe o seu próprio sucessor (prox_desc).
6. **Fim Enquanto**
7. **Se** o estado da própria referência for ATIVO, **Then Retorne** referência.
8. **Senão Retorne** NULO.
9. **Fim**

2. Primitiva de Sincronização: P (Solicitar)

Função: P(semáforo)

1. **Início**
2. **Se** o contador do semáforo (s) for maior que zero:
 - Decrementar o contador ($s = s - 1$).
3. **Senão (Recurso Indisponível):**
 - Definir o estado do processo atual como BLOQUEADO.
 - Inserir o processo atual no final da fila de espera do semáforo (Q).
 - proximo recebe o resultado de proximo_ativo_depois(atual).
 - antigo recebe atual.

- atual recebe proximo.
- Executar `transferência_contexto(antigo, atual)`.

4. **Fim Se**

5. **Fim**

3. Primitiva de Sincronização: V (Libertar)

Função: V(semáforo)

1. **Início**

2. **Se** a fila de espera do semáforo (Q) estiver vazia:

- Incrementar o contador do semáforo ($s = s + 1$).

3. **Senão (Existem processos à espera):**

- Remover o primeiro processo (p) da fila de espera (Q).
- Definir o estado de p como ATIVO.

4. **Fim Se**

5. **Fim**

4. Problema do Produtor/Consumidor (Estrutura do Projeto)

Variáveis Globais (Definições)

- **Constante TAM:** Tamanho máximo do buffer.
- **Vetor buffer:** Área de memória compartilhada.
- **Inteiros in e out:** Ponteiros (índices) do buffer para inserção e retirada.
- **Semáforos:** cheio, vazio e mutex.

Procedimento Depositar (Mensagem)

1. Inserir a mensagem na posição `buffer[in]`.
2. Atualizar o índice: $in = (in + 1) \% TAM$.

Procedimento Retirar (Mensagem)

1. Ler a mensagem da posição `buffer[out]`.
2. Atualizar o índice: $out = (out + 1) \% TAM$.

Lógica do Processo Produtor

1. **Repita Infinitamente:**

- Produção de uma nova mensagem.

- P(vazio) — *Aguarda slot livre (Sincronização).*
- P(mutex) — *Garante exclusão mútua.*
- **Executar: Depositar(mensagem).**
- V(mutex) — *Libera exclusão mútua.*
- V(cheio) — *Avisa que há item pronto (Sincronização).*
- yield() — *Cede a CPU voluntariamente.*

2. Até que falso

Lógica do Processo Consumidor

1. Repita Infinitamente:

- P(cheio) — *Aguarda item disponível (Sincronização).*
- P(mutex) — *Garante exclusão mútua.*
- **Executar: Retirar(mensagem).**
- V(mutex) — *Libera exclusão mútua.*
- V(vazio) — *Avisa que há slot livre (Sincronização).*
- Consumo/Exibição da mensagem processada.
- yield() — *Cede a CPU voluntariamente.*

2. Até que falso

5. Fluxo do Programa Principal (Main)

1. **Início**
2. Inicializar variáveis: in = 0, out = 0.
3. inicia_fila_prontos().
4. inicia_semaforo(cheio, 0).
5. inicia_semaforo(vazio, TAM).
6. inicia_semaforo(mutex, 1).
7. cria_processo(Produtor, "Produtor").
8. cria_processo(Consumidor, "Consumidor").
9. dispara_sistema().
10. **Fim**